

ARQUITETURAS DE COMPUTADORES

TURMA A1

TÓPICO 2 – MICROARQUITETURA (AULA 13)

Prof. Vinod Rebello, vinod@ic.uff.br, Sala 413

Tópico 2: A grande pergunta



Nossa ISA

ISA MAC-1 (nível 2)

- 23 Macroinstruções
- de 16 bits (coluna 1)
 - Opcode crescente (4, 8 ou 16 bits), não de tamanho fixo
 - Dois tipos de endereçamento:
 - Endereçamento direto
 - Endereçamento indexado
 - Um operando de 12 bits ou 8 bits ou sem operando
 - Não determina diretamente o número de dados necessário para a operação

Binary	Mnemonic	Instruction	Meaning
0000 xxxxxxxxxxxx	LODD	Load direct	$ac := m[x]$
0001 xxxxxxxxxxxx	STOD	Store direct	$m[x] := ac$
0010 xxxxxxxxxxxx	ADDD	Add direct	$ac := ac + m[x]$
0011 xxxxxxxxxxxx	SUBD	Subtract direct	$ac := ac - m[x]$
0100 xxxxxxxxxxxx	JPOS	Jump positive	If $ac \geq 0$ then $pc := x$
0101 xxxxxxxxxxxx	JZER	Jump zero	If $ac = 0$ then $pc := x$
0110 xxxxxxxxxxxx	JUMP	Jump	$pc := x$
0111 xxxxxxxxxxxx	LOCO	Load constant	$ac := x$ ($0 \leq x \leq 4095$)
1000 xxxxxxxxxxxx	LODL	Load local	$ac := m[sp + x]$
1001 xxxxxxxxxxxx	STOL	Store local	$m[x + sp] := ac$
1010 xxxxxxxxxxxx	ADDL	Add local	$ac := ac + m[sp + x]$
1011 xxxxxxxxxxxx	SUBL	Subtract local	$ac := ac - m[sp + x]$
1100 xxxxxxxxxxxx	JNEG	Jump negative	If $ac < 0$ then $pc := x$
1101 xx xxxxxxxx	JNZE	Jump nonzero	If $ac \neq 0$ then $pc := x$
1110 xx xxxxxxxx	CALL	Call procedure	$sp := sp - 1; m[sp] := pc; pc := x$
1111000 00000000	PSHI	Push indirect	$sp := sp - 1; m[sp] := m[ac]$
1111001 00000000	POPI	Pop indirect	$m[ac] := m[sp]; sp := sp + 1$
1111010 00000000	PUSH	Push onto stack	$sp := sp - 1; m[sp] := ac$
1111011 00000000	POP	Pop from stack	$ac := m[sp]; sp := sp + 1$
1111100 00000000	RETN	Return	$pc := m[sp]; sp := sp + 1$
1111101 00000000	SWAP	Swap ac, sp	$tmp := ac; ac := sp; sp := tmp$
1111110 yyyyyyyy	INSP	Increment sp	$sp := sp + y$ ($0 \leq y \leq 255$)
1111111 yyyyyyyy	DESP	Decrement sp	$sp := sp - y$ ($0 \leq y \leq 255$)

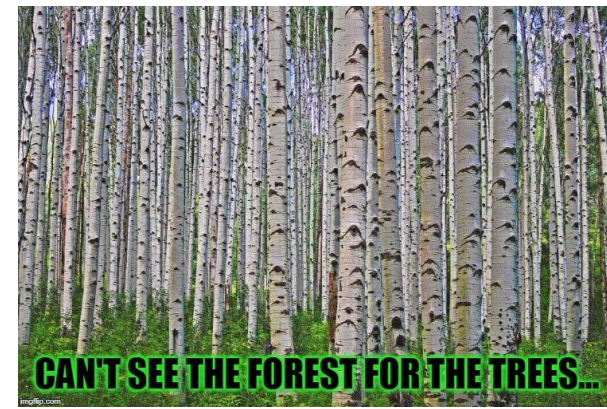
xxxxxxxxxxxx is a 12-bit machine address; in column 4 it is called x .
 yyyyyyyy is an 8-bit constant; in column 4 it is called y .

Exercícios de Tópico 1

- Considere um computador com interpretadores idênticos nos níveis 1, 2 e 3. Cada interpretador gasta n instruções para buscar, examinar e executar uma instrução.
 - Se uma instrução de nível 1 leva k segundos de execução, **quanto tempo uma instrução** de nível 2, 3 e 4 levará?
 - Esta questão já foi discutida

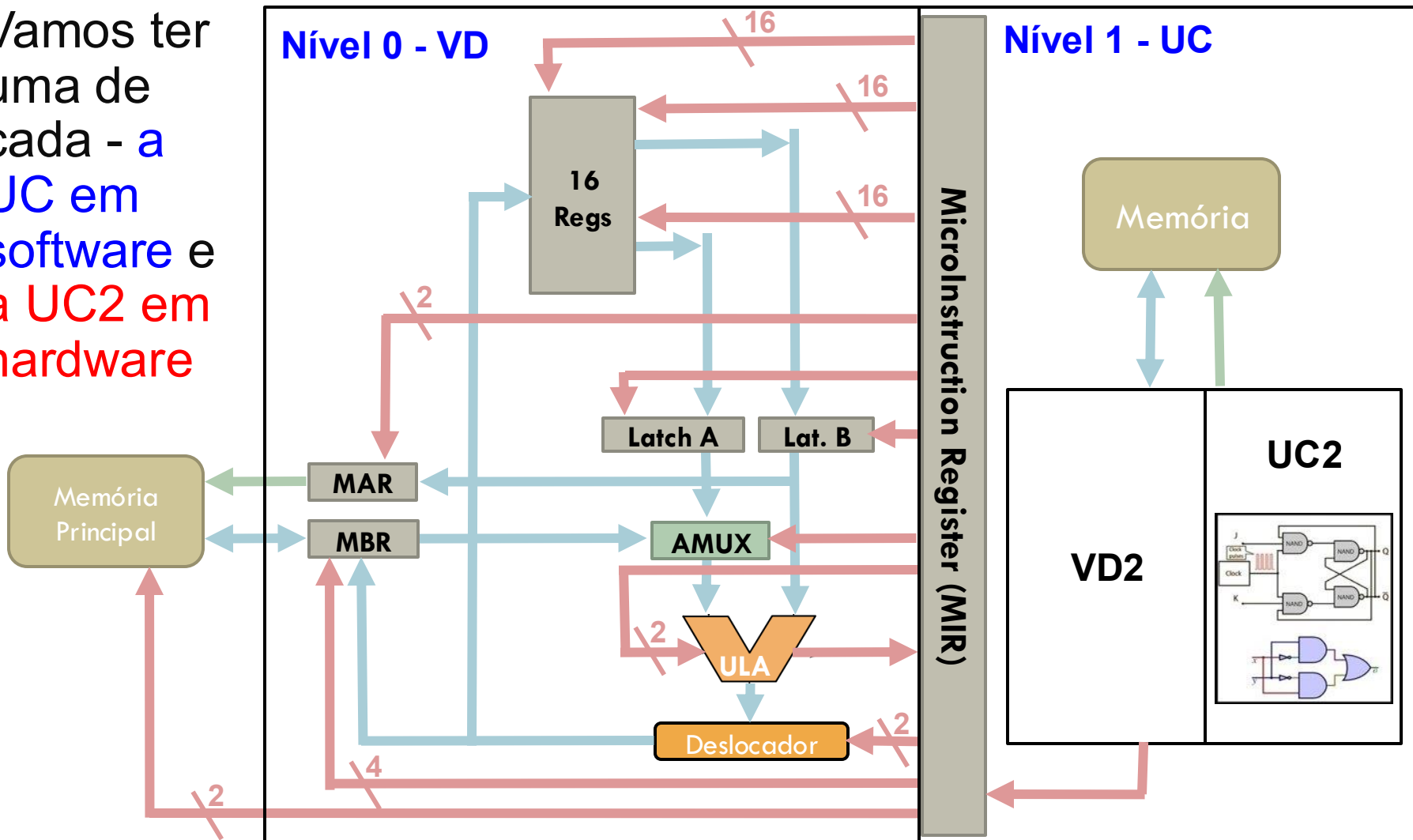
Exercícios de Tópico 1

- Considere que uma instrução do nível n pode fazer o trabalho de b instruções do nível $n-1$, ou seja, uma instrução no nível n é b vezes mais poderosa do que uma instrução no nível $n-1$. Se um **programa** de nível 1 requer s segundos para ser executado, quanto tempo **programas equivalentes** levariam nos níveis 2 e 3, assumindo que p instruções no nível n são necessárias para interpretar uma única instrução no nível $n+1$?
- É uma pergunta simples, mas exemplifica a dificuldade que muitos tem
 - de ler e entender o que está sendo solicitado e
 - de não ver a floresta por causa das árvores!



Unidade de Controle (UC) - Software

Vamos ter
uma de
cada - a
UC em
software e
a UC2 em
hardware



Projetando as Unidades de Controle

□ O que estará envolvido?

□ A geração

- da sequência correta dos valores de cada sinal de controle
- na hora certa, em relação aos outros sinais

□ Sequenciamento

□ Temporização

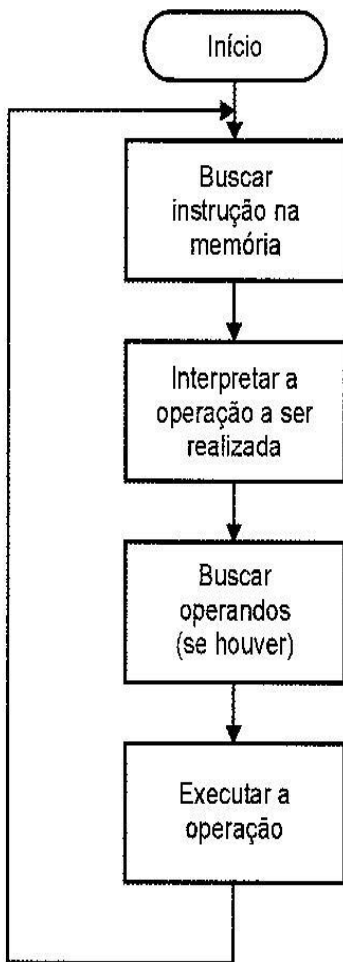


Figura 2.6 Ciclo básico de instrução.

Otimizações de UC x Programação

AMux	ULA	DESL	MBR	MAR	RD	WR	Lat. A	Lat. B	Bar. C	Bar. B	Bar. A
1	2	2	4	2	1	1	1	1	16	16	16
1	2	2	3	1	1	1	1	1	16	16	16
1	2	2	1	1	1	1	1	1	16	16	16
1	2	2	1	1	1	1	1	1	5	4	4

- ❑ **Passo 1:** Remove da UC, sinais cujo valores não mudam
- ❑ **Passo 2:** Unificar sinais com comportamentos parecidos
- ❑ **Passo 3:** Codificar o valor dos sinais
- ❑ **Passo 4:** Alguns dos sinais tem um comportamento cíclico ou repetitivo?

A microinstrução de MIC-1

AMux	ULA	DESL	MBR	MAR	RD	WR	Lat. A	Lat. B	Bar. C	Bar. B	Bar. A
1	2	2	4	2	1	1	1	1	16	16	16
1	2	2	3	1	1	1	1	1	16	16	16
1	2	2	1	1	1	1	1	1	16	16	16
1	2	2	1	1	1	1	1	1	5	4	4

Passo 4: Alguns dos sinais tem um comportamento cíclico ou repetitivo?

- O próprio modelo Von Neumann é cíclico, não é?

A microinstrução de MIC-1

AMux	ULA	DESL	MBR	MAR	RD	WR	Lat. A	Lat. B	Bar. C	Bar. B	Bar. A
1	2	2	4	2	1	1	1	1	16	16	16
1	2	2	3	1	1	1	1	1	16	16	16
1	2	2	1	1	1	1	1	1	16	16	16
1	2	2	1	1	1	1	1	1	5	4	4

Passo 4: Alguns dos sinais tem um comportamento cíclico ou repetitivo?

- MVN Etapas 3 e 4 utiliza a VD para executar de uma operação simples
 - Por ex., como seria a implementação da operação $PC = PC + 1$?

A microinstrução de MIC-1

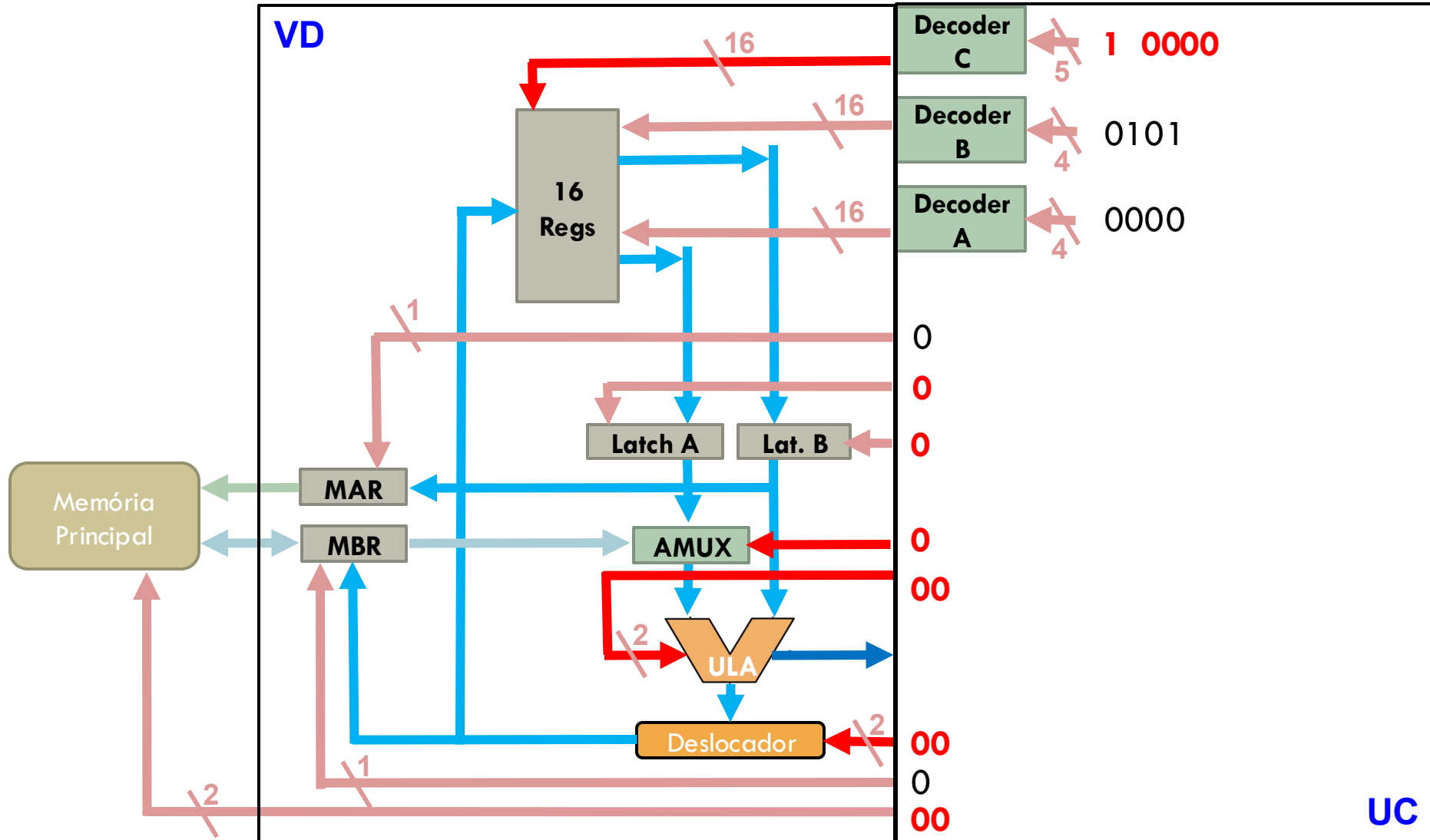
O valor dos sinais: em preto, não importa; em vermelho, é obrigatório

	AMux (1)	ULA (2)	DESL (2)	MBR (1)	MAR (1)	RD (1)	WR (1)	Lat. A (1)	Lat. B (1)	En C (1)	Bar. C (4)	Bar. B (4)	Bar. A (4)
a)	0	00	00	0	0	0	0	0	0	0	0000	0110	0000
b)	0	00	00	0	0	0	0	1	1	0	0000	0110	0000
c)	0	00	00	0	0	0	0	0	0	1	0000	0110	0000

A realização da operação $PC = PC + 1$

- a) Buscar os operandos nos registradores
- b) Depois de um tempo t_{BO} , pode habilitar as entradas dos *latches* A e B
- c) Depois de um tempo t_{EXE} a mais, pode gravar o resultado no PC

Projetando a Unidade de Controle



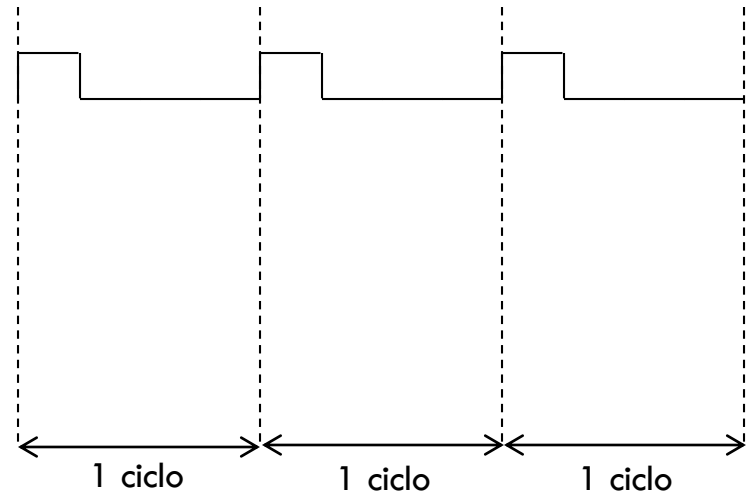
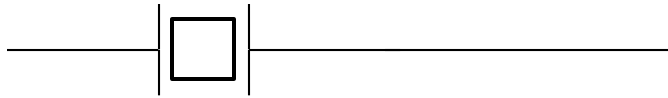
A microinstrução de MIC-1

O valor dos sinais: em preto, não importa; em vermelho, é obrigatório

	AMux (1)	ULA (2)	DESL (2)	MBR (1)	MAR (1)	RD (1)	WR (1)	Lat. A (1)	Lat. B (1)	En C (1)	Bar. C (4)	Bar. B (4)	Bar. A (4)
a)	0	00	00	0	0	0	0	0	0	0	0000	0110	0000
b)	0	00	00	0	0	0	0	1	1	0	0000	0110	0000
c)	0	00	00	0	0	0	0	0	0	1	0000	0110	0000

- Para realizar qualquer operação do tipo registrador-operação-registrador como $PC = PC + 1$
 - Precisamos de três microinstruções, isto é, 72 bits em total!
 - A maioria dos bits nem muda de valor – seria uma mau utilização de espaço
- Vamos remover da memória os sinais de controle que mudam
 - Porque estas mudanças sempre ocorrem no mesmo instante em relação de a)

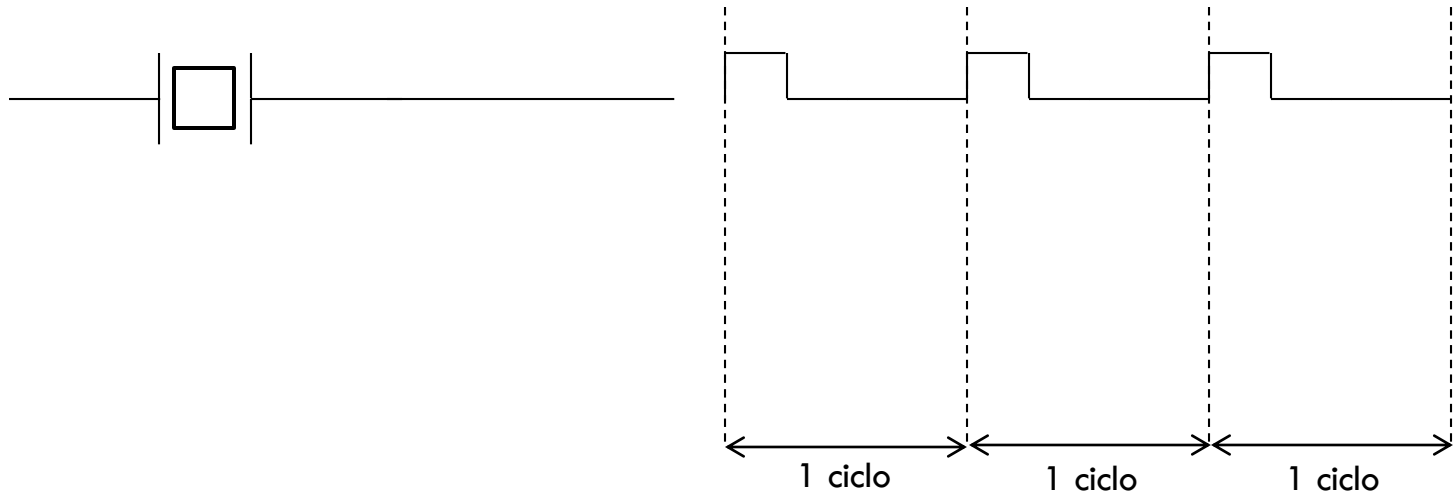
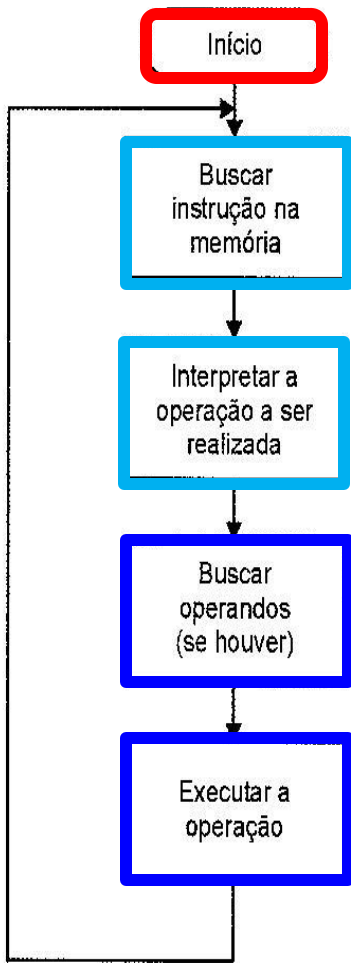
O relógio de MIC-1



- O relógio é componente que gerar um sinal de onda de frequência constante
 - Cada período se chama **um ciclo**
 - Um ciclo tem uma duração constante, predeterminada

O relógio de MIC-1

MVN2 p/ μ insts



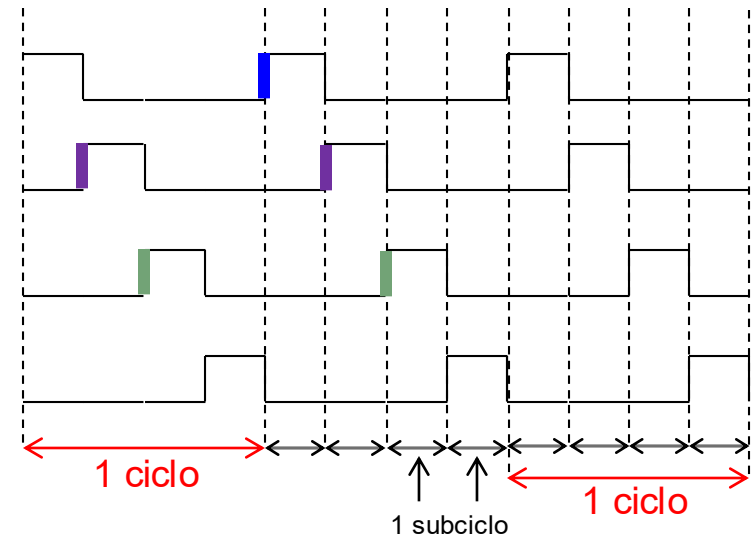
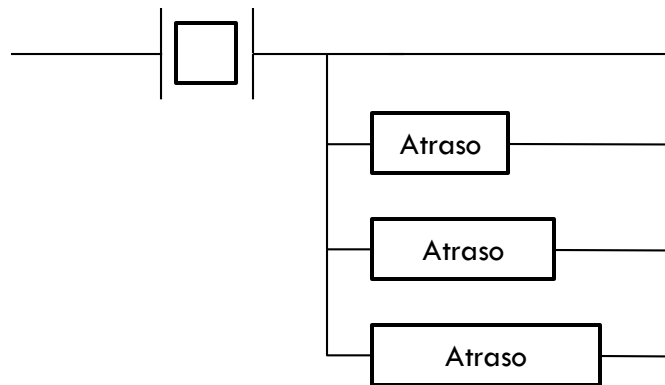
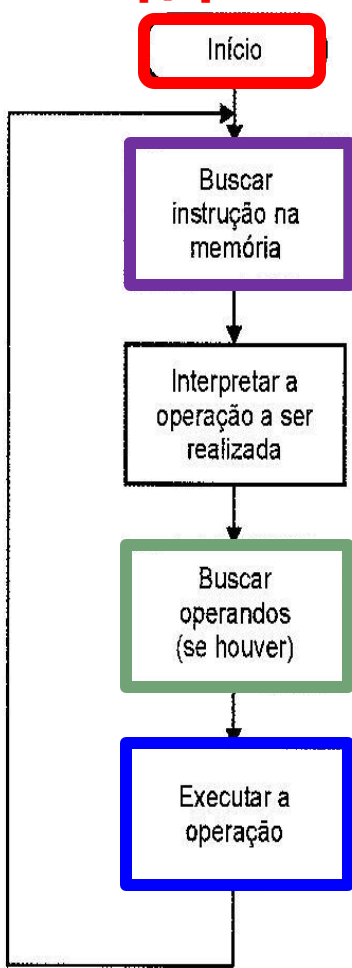
□ O modelo Von Neumann é cíclico!

- Se conseguimos remover os sinais (de temporização) que sempre precisam mudar *durante* cada iteração, vão precisar só **realizar** uma **microinstrução por ciclo**
- A execução do microprograma (segundo o MVN2) envolve a busca da microinstrução, a busca dos operandos, a execução da operação e a gravação do resultado, depois repete

Figura 2.6 Ciclo básico de instrução.

O relógio de MIC-1

MVN2 p/ μ insts



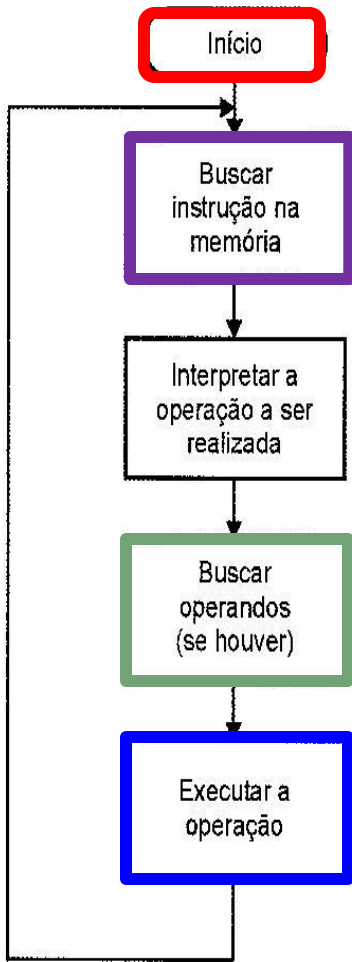
□ Podemos dividir o ciclo em 4 subciclos

- O relógio teria mais 3 sinais derivados do sinal original
- Cada um com pode ter um atraso diferente, por exemplo, t_{BO}

Figura 2.6 Ciclo básico de instrução.

O relógio de MIC-1

MVN2 p/ μ insts



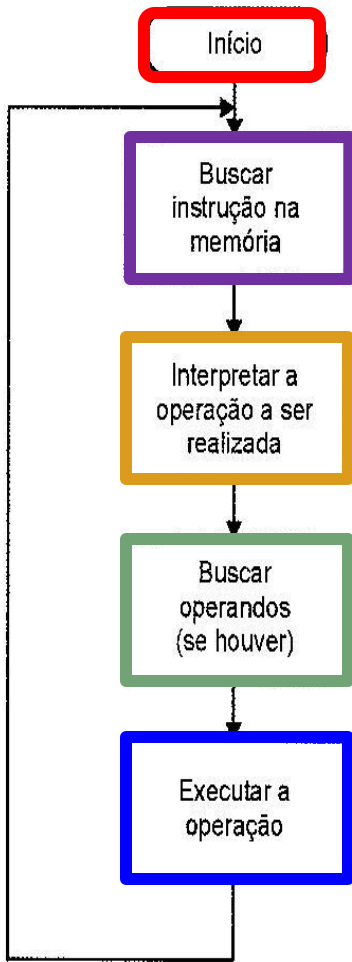
□ Com um ciclo dividido em 4 subciclos

1. O **primeiro subciclo** daria tempo suficiente para buscar a próxima microinstrução na memória de controle
2. Durante o **segundo subciclo**, seria buscado os operandos do conjunto de registradores, independente da necessidade real. No final do subciclo, serão capturados pelos *latches*
3. O **terceiro subciclo** daria tempo para o dado de barramento B chega na entrada do MAR, e realizar alguma operação no ULA
4. Até o final de **quarto subciclo**, o resultado do deslocador chegaria no MBR e/ou em um dos registradores

Figura 2.6 Ciclo básico de instrução.

O relógio de MIC-1

MVN2 p/ μ insts



??

- Com um ciclo dividido em 4 subciclos

1. O **primeiro subciclo** daria tempo suficiente para buscar a próxima microinstrução na memória de controle
2. Durante o **segundo subciclo**, seria buscado os operandos do conjunto de registradores, independente da necessidade real. No final do subciclo, serão capturados pelos *latches*
3. O **terceiro subciclo** daria tempo para o dado de barramento B chega na entrada do MAR, e realizar alguma operação no ULA
4. Até o final de **quarto subciclo**, o resultado do deslocador chegaria no MBR e/ou em um dos registradores

Figura 2.6 Ciclo básico de instrução.

O Ciclo de VN para um Microprograma

- O ciclo VN (realizado em **hardware**) para executar **uma microinstrução** é dividido em 4 **subciclos**:

1. O primeiro subciclo busca a próxima microinstrução na memória de controle
2. Durante o segundo subciclo, seria buscado operandos do conjunto de registradores, independente da necessidade real. No final do subciclo, serão capturados pelos *latches*
3. O terceiro subciclo daria tempo para o dado de barramento B chega na entrada do MAR para começar ler ou escrever na MP, e realizar alguma operação no ULA e o estado do resultando seja retornado a UC, ...
4. Até o final de quarto subciclo, o resultado do deslocador chegaria no MBR e/ou em um dos registradores

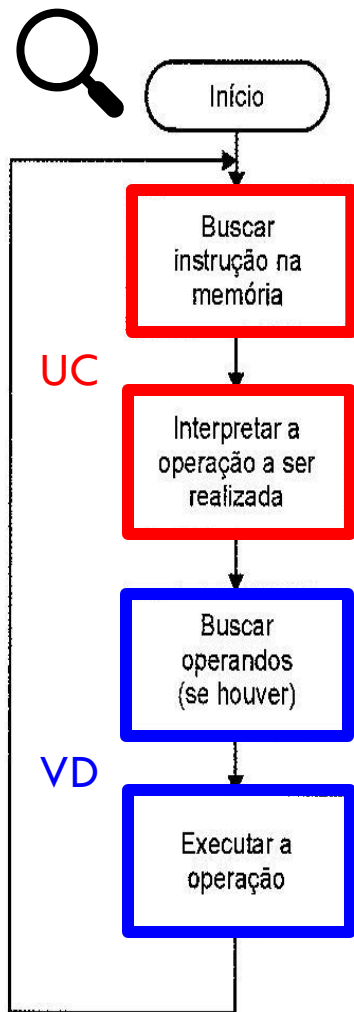
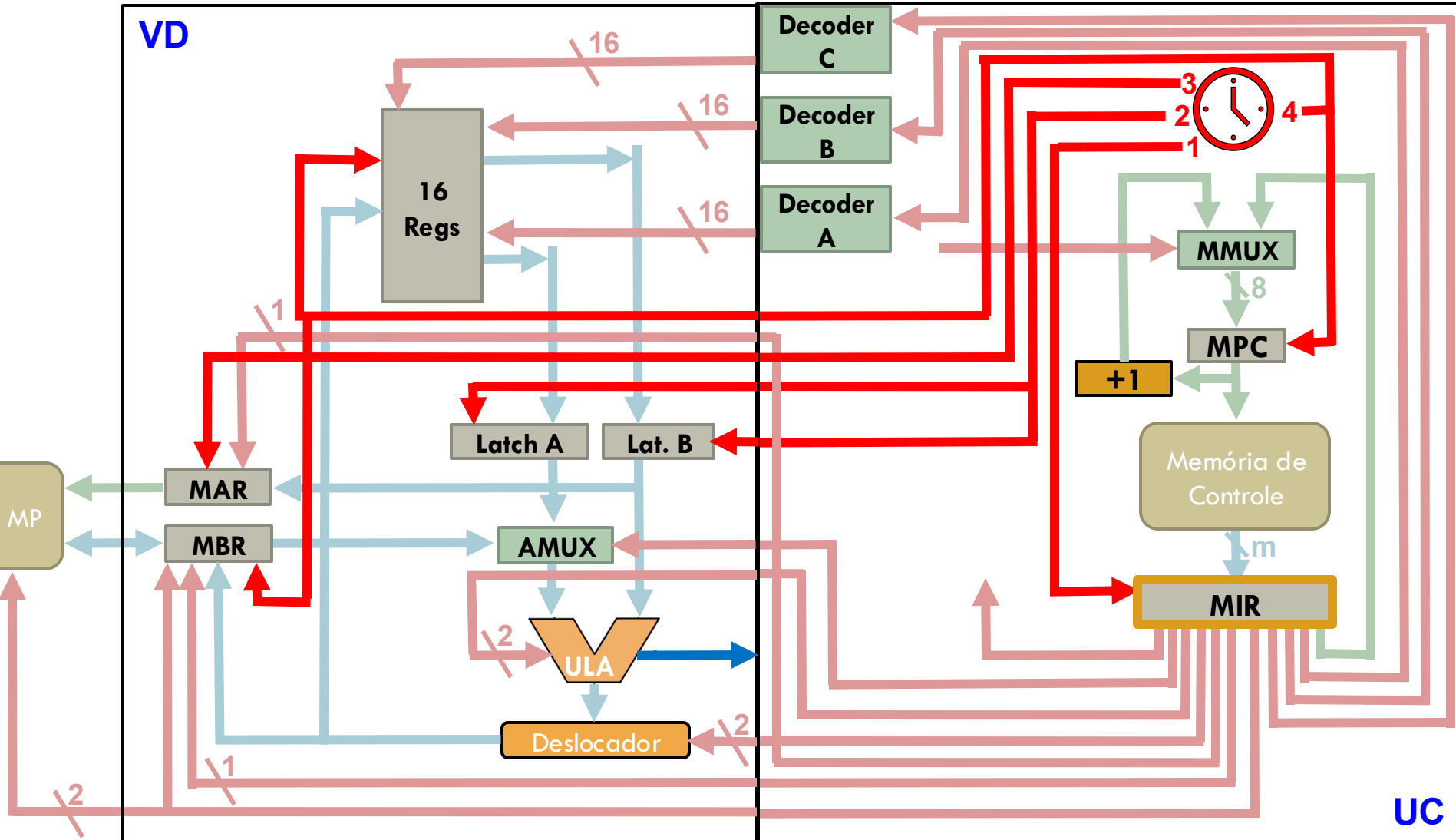


Figura 2.6 Ciclo básico de instrução.

Ciclo Von Neumann em hardware



A microinstrução de MIC-1

A microinstrução tem 32 bits

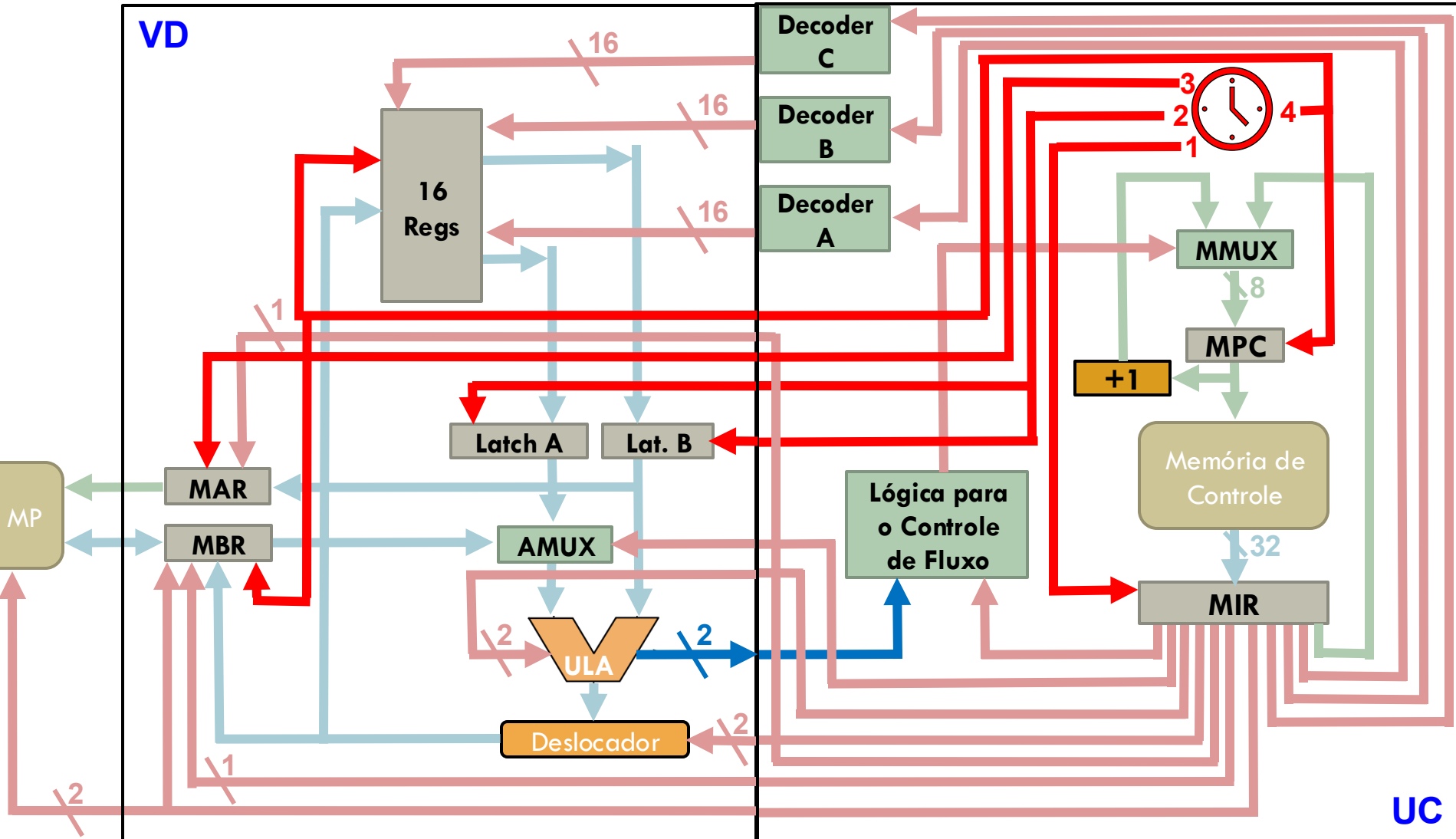


OPCODE

OPERANDOS

- A (micro)programa é **uma sequência** de (micro)instruções
 - Precisamos realizar **desvios condicionais** e **incondicionais**:
 - Se COND = 00, não desvia; Se COND = 01 e N = 1, desvia; Se COND = 10 e Z = 1, desvia; Se COND = 11, desvia para o endereço indicado no ADDR.
 - Os bits de status da ULA: N identifica se o resultado da ULA é negativo e Z indica que o resultado da ULA é zero
 - **Nesta versão de MIC-1 não tratamos estorno (overflow) nos resultados da soma na ULA**
- Com 12 bits de opcode, parece que tem 4096 possíveis microinstruções em L1...

O projeto do microprocessador MIC-1

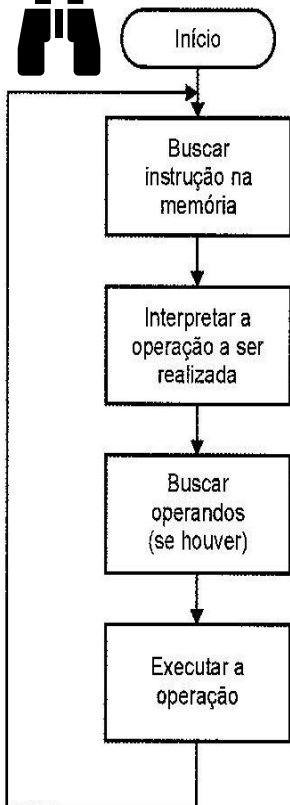


O Ciclo de VN para o Macroprograma

OPCODE

OPERANDOS

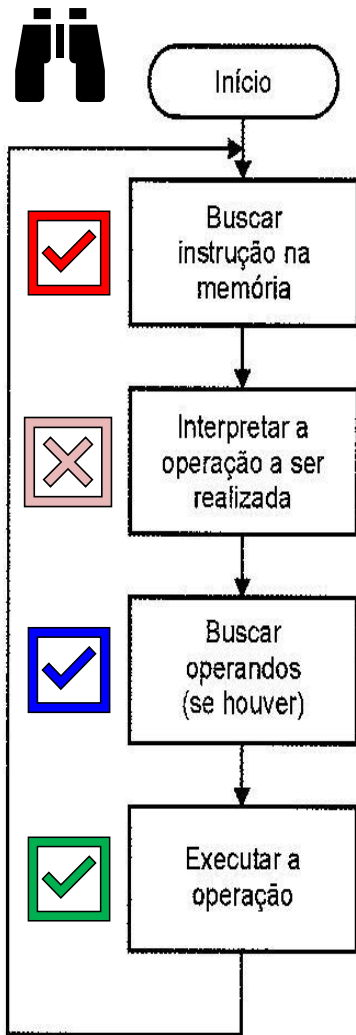
AMux (1)	COND (2)	ULA (2)	DESL (2)	MBR (1)	MAR (1)	RD (1)	WR (1)	En C (1)	Bar. C (4)	Bar. B (4)	Bar. A (4)	ADDR (8)



- O **μprograma** tem que interpretar o **macroprograma**, isto é, implementa o modelo VN do macroprograma
- Por ser um programa, o ciclo VN seria realizada em **software**
 - Busca a próxima **macroinstrução**
 - Identifica a macroinstrução buscada
 - Executa uma sequência de **microinstruções** equivalente a macroinst
 - Repete

Figura 2.6 Ciclo básico de instrução.

O Microprograma de MIC-1



- O **microprograma** tem que interpretar cada **macroprograma**
 - Busca a próxima macroinstrução
 - Identifica a macroinstrução buscada
 - Executa uma sequência de microinstruções equivalente
 - Repete
- Já temos visto como usar o Via de Dados
- A UC2 determina que acontece, e quando, durante num ciclo
- O **microprograma** define que acontece ciclo a ciclo
- A interpretação ou realização de uma macroinstrução leva vários ciclos (requer a execução de várias microinstruções)
- **Por quê?**

Figura 2.6 Ciclo básico de instrução.

Um Microprograma para MIC-1

- Este microprograma tem 79 microinstruções de 32 bits

0: mar:=pc; rd;	Busca	40: tir:=lshift(tir); if n then goto 46;	
1: pc:=pc + 1; rd;	Busca	41: alu:=tir; if n then goto 44;	
2: jr:=mbr; if n then goto 28;	Busca/Identifica	42: alu:=ac; if n then goto 22;	{1100 = JNEG}
3: tir:=lshift(ir + jr); if n then goto 19;	Identifica	43: goto 0;	
4: tir:=lshift(tir); if n then goto 11;	Identifica	44: alu:=ac; if z then goto 0;	{1101 = JNZE}
5: alu:=tir; if n then goto 9;	Identifica	45: pc:=band(ir.amask); goto 0;	
6: mar:=ir; rd;	Executa	46: tir:=lshift(tir); if n then goto 50;	
	{0000 = LODD}	47: sp:=sp + (-1);	{1110 = CALL}
7: rd;	Executa	48: mar:=sp; mbr:=pc; wr;	
8: ac:=mbr; goto 0;	Executa	49: pc:=band(ir.amask); wr; goto 0;	
9: mar:=ir; mbr:=ac; wr;	{0001 = STOD}	50: tir:=lshift(tir); if n then goto 65;	
10: wr; goto 0;	Executa	51: tir:=lshift(tir); if n then goto 59;	
11: alu:=tir; if n then goto 15;	Identifica	52: alu:=tir; if n then goto 56;	
12: mar:=ir; rd;	{0010 = ADDD}	53: mar:=ac; rd;	{1111-0000 = PSHI}
13: rd;	Executa	54: sp:=sp + (-1); rd;	
14: ac:=mbr + ac; goto 0;	Executa	55: mar:=sp; wr; goto 10;	
15: mar:=ir; rd;	{0011 = SUBD}	56: mar:=sp; sp:=sp + 1; rd;	{1111-0010 = POPI}
16: ac:=ac + 1; rd;	Executa	57: rd;	
17: a:=inv(mbr);	Executa	58: mar:=ac; wr; goto 10;	
18: ac:=ac + a; goto 0;	Executa	59: alu:=tir; if n then goto 62;	
19: tir:=lshift(tir); if n then goto 25;	Identifica	60: sp:=sp + (-1);	{1111-0100 = PUSH}
20: alu:=tir; if n then goto 23;		61: mar:=sp; mbr:=ac; wr; goto 10;	
21: alu:=ac; if n then goto 0;	{0100 = JPOS}	62: mar:=sp; sp:=sp + 1; rd;	{1111-0110 = POP}
22: pc:=band(ir.amask); goto 0;		63: rd;	
23: alu:=ac; if z then goto 22;	{0101 = JZER}	64: ac:=mbr; goto 0;	
24: goto 0;		65: tir:=lshift(tir); if n then goto 73;	
25: alu:=tir; if n then goto 27;		66: alu:=tir; if n then goto 70;	
26: pc:=band(ir.amask); goto 0;	{0110 = JUMP}	67: mar:=sp; sp:=sp + 1; rd;	{1111-1000 = RETN}
27: ac:=band(ir.amask); goto 0;	{0111 = LOCO}	68: rd;	
28: tir:=lshift(ir + jr); if n then goto 40;		69: pc:=mbr; goto 0;	
29: tir:=lshift(tir); if n then goto 35;		70: a:=ac;	{1111-1010 = SWAP}
30: alu:=tir; if n then goto 33;		71: ac:=sp;	
31: a:=ir + sp;	{1000 = LODL}	72: sp:=a; goto 0;	
32: mar:=a; rd; goto 7;		73: alu:=tir; if n then goto 76;	
33: a:=ir + sp;	{1001 = STOL}	74: a:=band(ir.smask);	{1111-1100 = INSP}
34: mar:=a; mbr:=ac; wr; goto 10;		75: sp:=sp + a; goto 0;	
35: alu:=tir; if n then goto 38;		76: a:=band(ir.smask);	{1111-1110 = DESP}
36: a:=ir + sp;	{1010 = ADDL}	77: a:=inv(a);	
37: mar:=a; rd; goto 13;		78: a:=a + 1; goto 75;	
38: a:=ir + sp;	{1011 = SUBL}		
39: mar:=a; rd; goto 16 ;			

Busca na memória de MIC-1

	OPCODE									OPERANDOS			
	AMux (1)	COND (2)	ULA (2)	DESL (2)	MBR (1)	MAR (1)	RD (1)	WR (1)	En C (1)	Bar. C (4)	Bar. B (4)	Bar. A (4)	ADDR (8)
0:	0	00	00	00	0	1	1	0	0	0000	0000	0000	(01) ₁₆

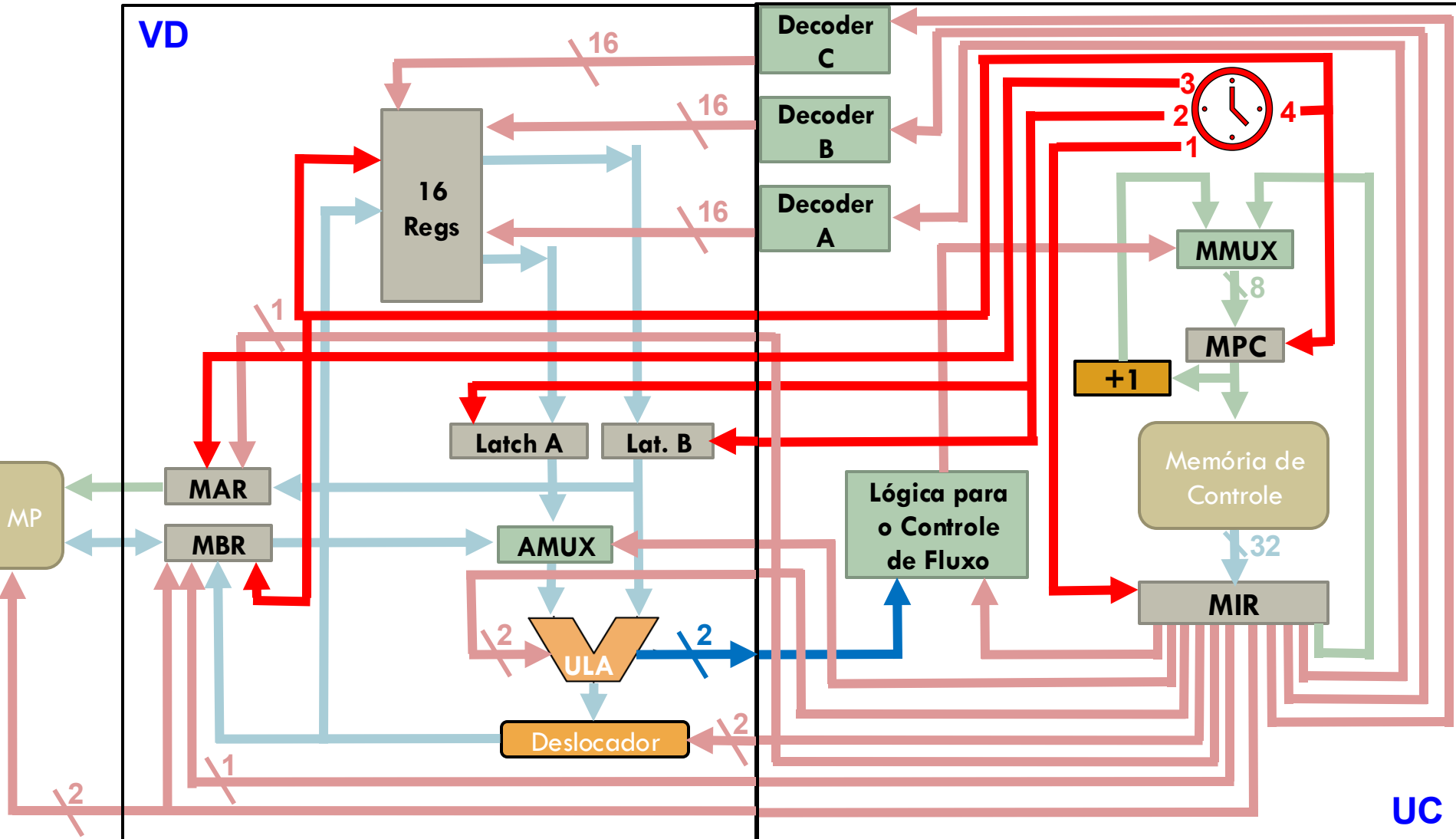


- Dado que a latência da "memória principal" é seis subciclos
- $IR = mp[PC];$
 - 0: $mar := pc; rd;$
 - 1: $pc := pc + 1; rd;$
 - 2: $ir := mbr;$

C	SC	Ação
1	1	Busca da $\mu inst$
1	2	Busca $pc, rd=1$
1	3	End = ??, $rd=1$
1	4	$mar := pc; rd=1$
2	1	Busca da $\mu inst$; $rd=1$
2	2	Busca $pc, 1$; $rd=1$
2	3	Soma; $rd=1$
2	4	Soma; $rd=1$
3	1	Busca da $\mu inst$; $rd=1$
3	2	Amux=1; EnC=1; $rd=0$

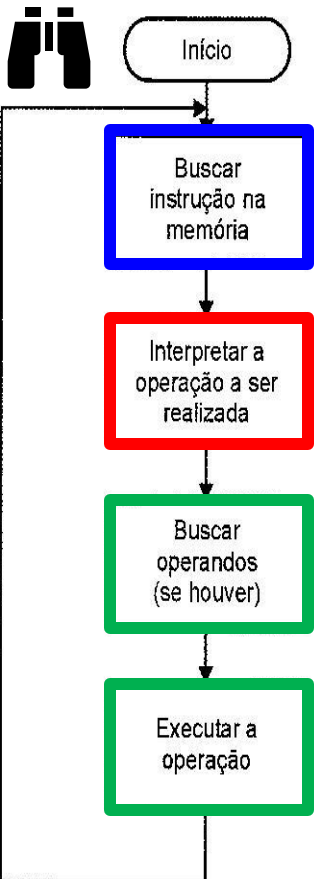
Figura 2.6 Ciclo básico de instrução.

O projeto do microprocessador MIC-1



Identificando a macroinstrução

OPCODE									OPERANDOS			
AMux (1)	COND (2)	ULA (2)	DESL (2)	MBR (1)	MAR (1)	RD (1)	WR (1)	En C (1)	Bar. C (4)	Bar. B (4)	Bar. A (4)	ADDR (8)
3: 0	01	00	10	0	0	0	0	1	0100	0011	0011	(13) ₁₆



□ Macroinstrução tem 16 bits:

- 2: $ir := mbr$; if n then goto 28;
- 3: $tir := lshift(ir + ir)$; if n then goto 19;
- 4: $tir := lshift(tir)$; if n then goto 11;
- 5: $alu := tir$; if n then goto 9;
- 6: $mar := ir$; rd;
- 7: rd;
- 8: $ac := mbr$; goto 0;
- 9: $mar := ir$; $mbr := ac$; wr;
- 10: wr; goto 0;

Interpreta

A instrução é negativa?

Executa {0000 = LODD}

Executa

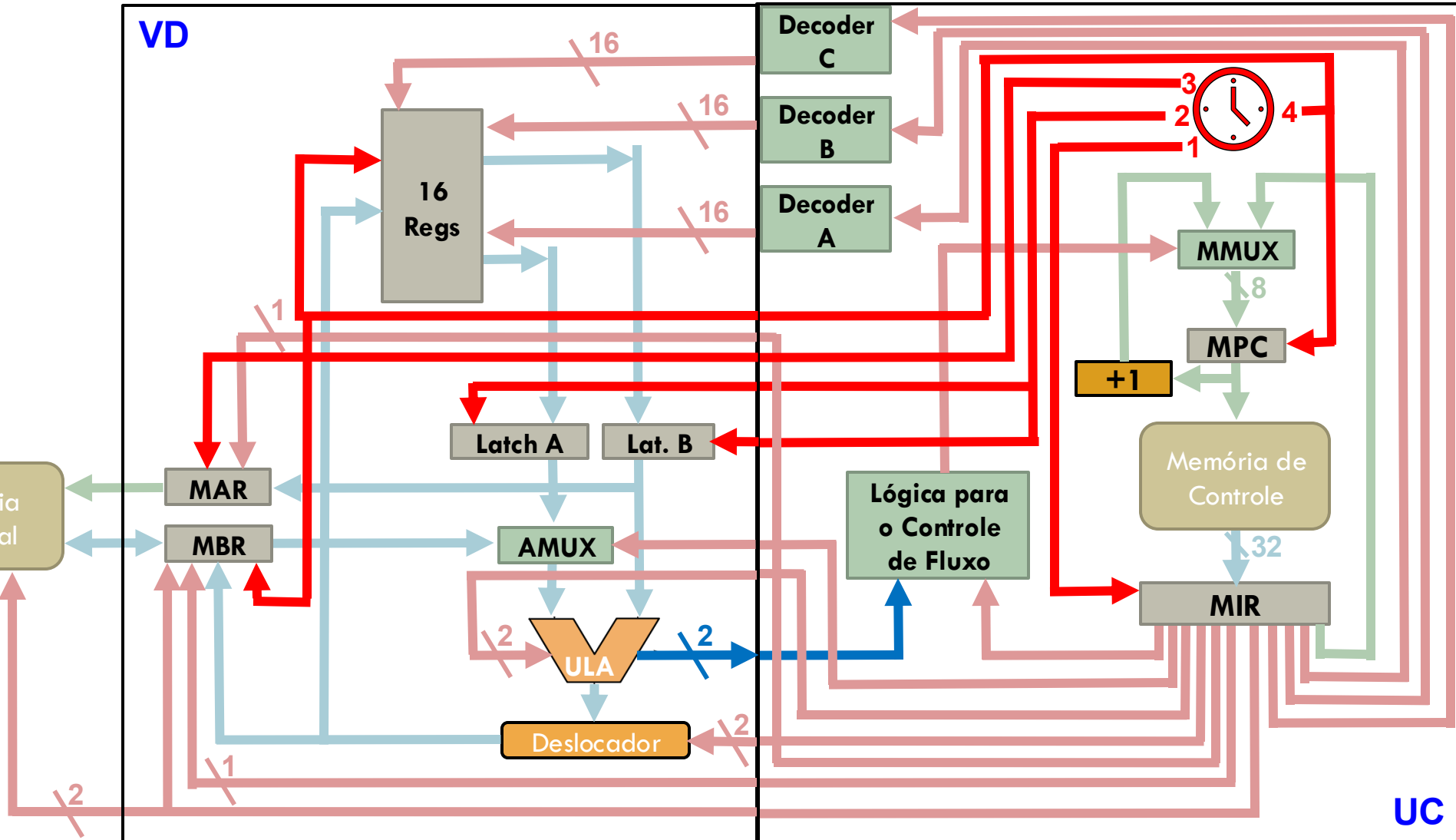
Executa

Executa {0001 = STOD}

Executa

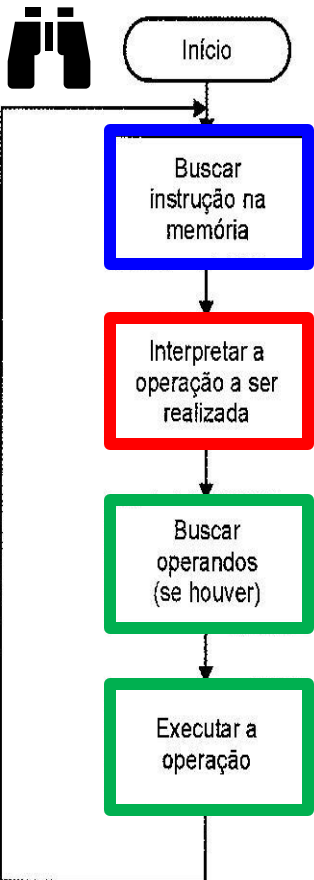
Figura 2.6 Ciclo básico de instrução.

O projeto do microprocessador MIC-1



Identificando a macroinstrução

OPCODE									OPERANDOS			
AMux (1)	COND (2)	ULA (2)	DESL (2)	MBR (1)	MAR (1)	RD (1)	WR (1)	En C (1)	Bar. C (4)	Bar. B (4)	Bar. A (4)	ADDR (8)
3: 0	01	00	10	0	0	0	0	1	0100	0011	0011	13



□ Macroinstrução tem 16 bits: ABCDEFGHIJKLMNOP

- 2: *ir* := *mbr*; if *n* then goto 28; *if A = 1 then goto 28*
- 3: *tir* := *lshift*(*ir* + *ir*); if *n* then goto 19; *if B = 1 then goto 19*
- 4: *tir* := *lshift*(*tir*); if *n* then goto 11; *if C = 1 then goto 11*
- 5: *alu* := *tir*; if *n* then goto 9; *if D = 1 then goto 9*
- 6: *mar* := *ir*; *rd*; Executa {0000 = LODD}
- 7: *rd*; Executa
- 8: *ac* := *mbr*; goto 0; Executa
- 9: *mar* := *ir*; *mbr* := *ac*; *wr*; Executa {0001 = STOD}
- 10: *wr*; goto 0; Executa

Figura 2.6 Ciclo básico de instrução.

Identificando a macroinstrução

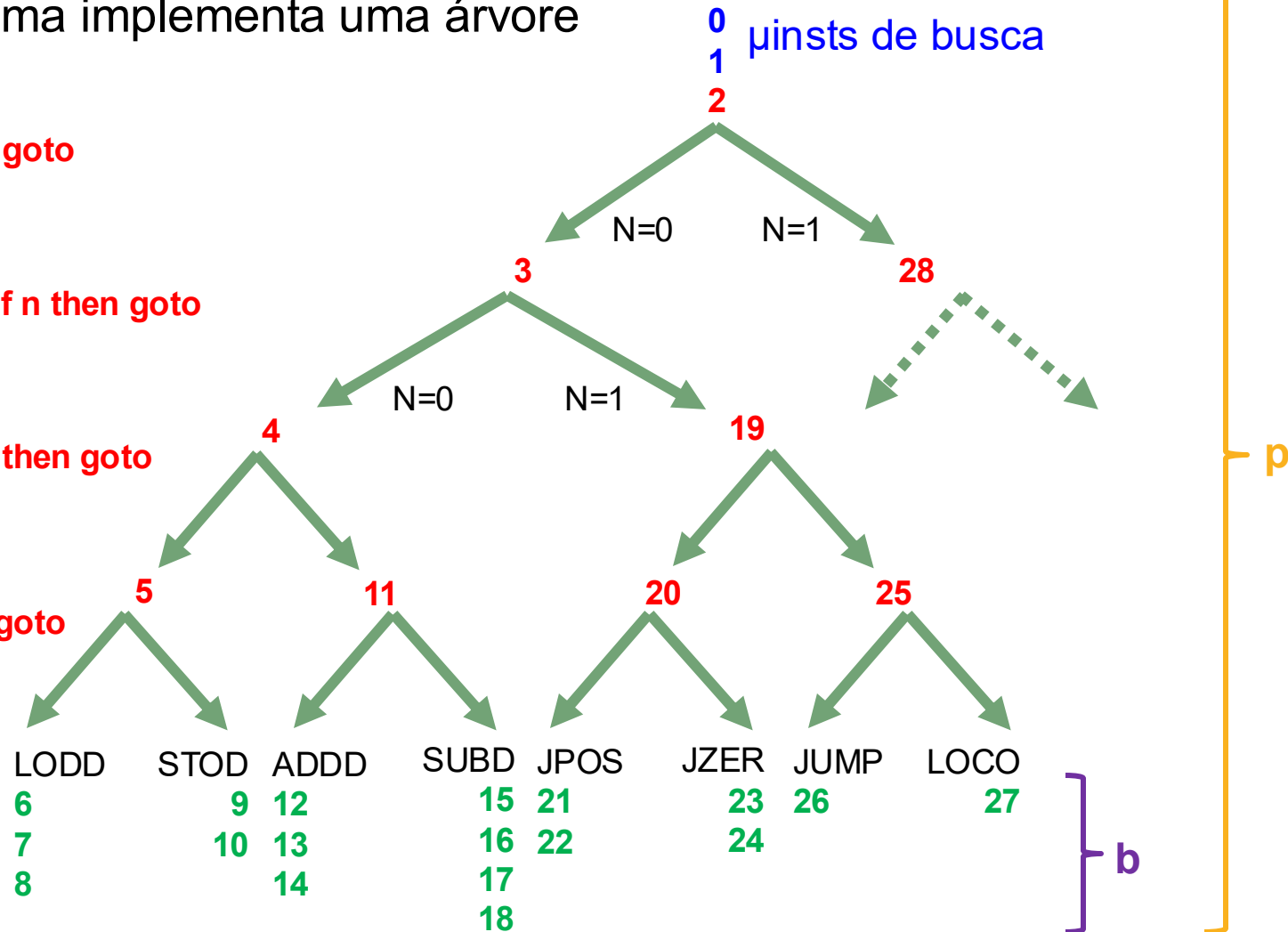
- O microprograma implementa uma árvore de decisão:

ir := mbr; if n then goto

tir := lshift(ir + ir); if n then goto

tir := lshift(tir); if n then goto

alu := tir; if n then goto



O Microprograma de MIC-1

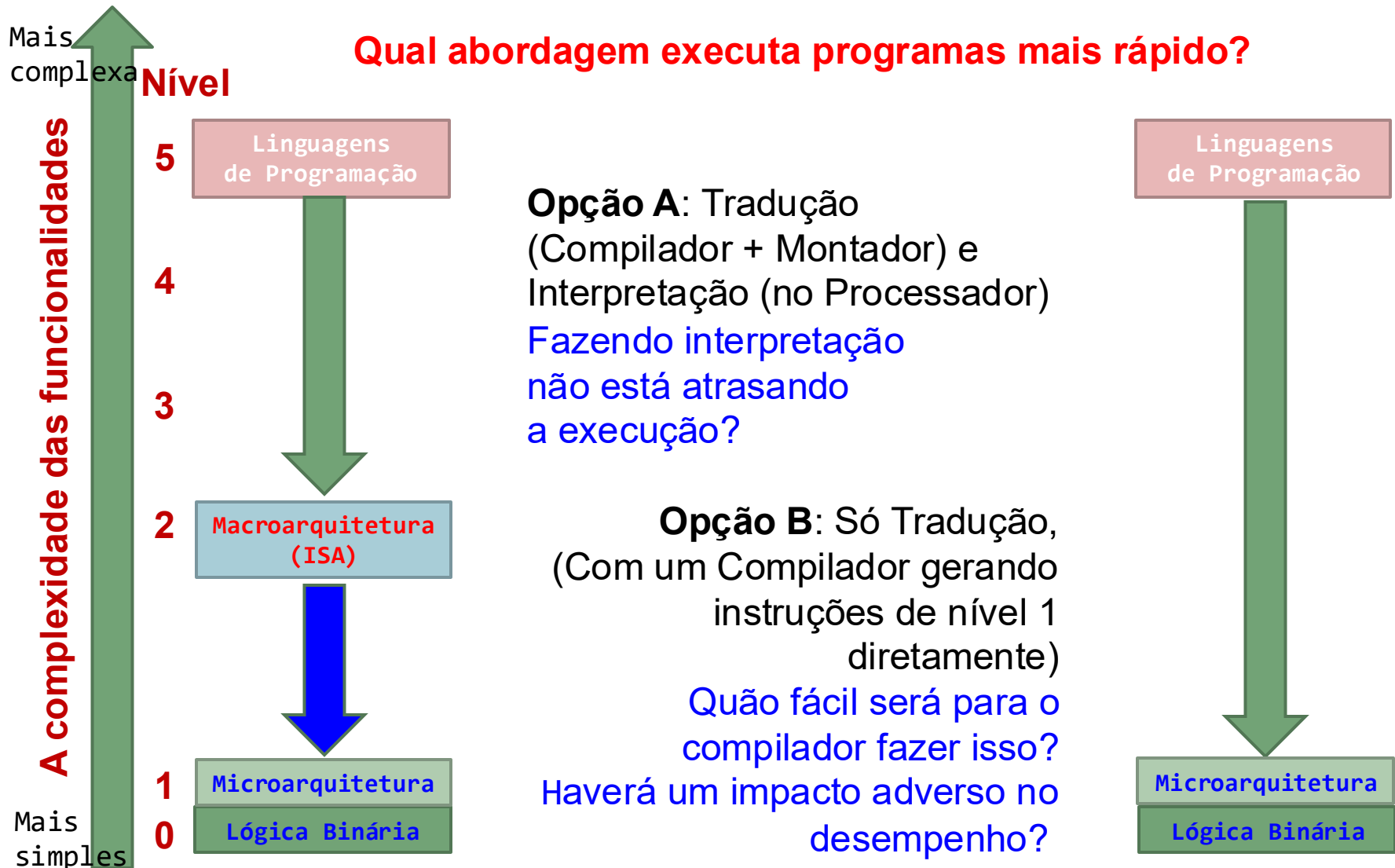
- Este microprograma tem 79 microinstruções de 32 bits

0: mar:=pc; rd;	Busca	40: tir:=lshift(tir); if n then goto 46;	
1: pc:=pc + 1; rd;	Busca	41: alu:=tir; if n then goto 44;	
2: jr:=mbr; if n then goto 28;	Busca/Identifica	42: alu:=ac; if n then goto 22;	{1100 = JNEG}
3: tir:=lshift(ir + jr); if n then goto 19;	Identifica	43: goto 0;	
4: tir:=lshift(tir); if n then goto 11;	Identifica	44: alu:=ac; if z then goto 0;	{1101 = JNZE}
5: alu:=tir; if n then goto 9;	Identifica	45: pc:=band(ir.amask); goto 0;	
6: mar:=ir; rd;	Executa	46: tir:=lshift(tir); if n then goto 50;	
	{0000 = LODD}	47: sp:=sp + (-1);	{1110 = CALL}
7: rd;	Executa	48: mar:=sp; mbr:=pc; wr;	
8: ac:=mbr; goto 0;	Executa	49: pc:=band(ir.amask); wr; goto 0;	
9: mar:=ir; mbr:=ac; wr;	{0001 = STOD}	50: tir:=lshift(tir); if n then goto 65;	
10: wr; goto 0;	Executa	51: tir:=lshift(tir); if n then goto 59;	
11: alu:=tir; if n then goto 15;	Identifica	52: alu:=tir; if n then goto 56;	
12: mar:=ir; rd;	{0010 = ADDD}	53: mar:=ac; rd;	{1111-0000 = PSHI}
13: rd;	Executa	54: sp:=sp + (-1); rd;	
14: ac:=mbr + ac; goto 0;	Executa	55: mar:=sp; wr; goto 10;	
15: mar:=ir; rd;	{0011 = SUBD}	56: mar:=sp; sp:=sp + 1; rd;	{1111-0010 = POPI}
16: ac:=ac + 1; rd;	Executa	57: rd;	
17: a:=inv(mbr);	Executa	58: mar:=ac; wr; goto 10;	
18: ac:=ac + a; goto 0;	Executa	59: alu:=tir; if n then goto 62;	
19: tir:=lshift(tir); if n then goto 25;	Identifica	60: sp:=sp + (-1);	{1111-0100 = PUSH}
20: alu:=tir; if n then goto 23;		61: mar:=sp; mbr:=ac; wr; goto 10;	
21: alu:=ac; if n then goto 0;	{0100 = JPOS}	62: mar:=sp; sp:=sp + 1; rd;	{1111-0110 = POP}
22: pc:=band(ir.amask); goto 0;		63: rd;	
23: alu:=ac; if z then goto 22;	{0101 = JZER}	64: ac:=mbr; goto 0;	
24: goto 0;		65: tir:=lshift(tir); if n then goto 73;	
25: alu:=tir; if n then goto 27;		66: alu:=tir; if n then goto 70;	
26: pc:=band(ir.amask); goto 0;	{0110 = JUMP}	67: mar:=sp; sp:=sp + 1; rd;	{1111-1000 = RETN}
27: ac:=band(ir.amask); goto 0;	{0111 = LOCO}	68: rd;	
28: tir:=lshift(ir + jr); if n then goto 40;		69: pc:=mbr; goto 0;	
29: tir:=lshift(tir); if n then goto 35;		70: a:=ac;	{1111-1010 = SWAP}
30: alu:=tir; if n then goto 33;		71: ac:=sp;	
31: a:=ir + sp;	{1000 = LODL}	72: sp:=a; goto 0;	
32: mar:=a; rd; goto 7;		73: alu:=tir; if n then goto 76;	
33: a:=ir + sp;	{1001 = STOL}	74: a:=band(ir.smask);	{1111-1100 = INSP}
34: mar:=a; mbr:=ac; wr; goto 10;		75: sp:=sp + a; goto 0;	
35: alu:=tir; if n then goto 38;		76: a:=band(ir.smask);	{1111-1110 = DESP}
36: a:=ir + sp;	{1010 = ADDL}	77: a:=inv(a);	
37: mar:=a; rd; goto 13;		78: a:=a + 1; goto 75;	
38: a:=ir + sp;	{1011 = SUBL}		
39: mar:=a; rd; goto 16 ;			

O Microprograma de MIC-1

- A memória de controle tem 256×32 bits, isto é, 8K bits; este microprograma ocupa apenas $79 \times 32 = 2528$ bits.
 - Mas não é a única implementação
 - Têm possibilidades para reduzir o tamanho da microinstrução sem perder funcionalidade ... (vai pensando nisso)
- Na *média* 9 ciclos (μ insts) para executar uma macroinstrução
- Identificar cada macroinstrução tem um *custo* significativo
 - Será que tem uma maneira para reduzir este custo?
 - Em hardware, é muito provável. Como programador, você faria o que?
- MIC-1 (ainda) não é uma implementação otimizada em termos de desempenho e custo

Tópico 2: A grande pergunta



Se $(i \leq n)$ então $m := a[i];$

- Você tem o papel de ser um compilador
- (Opção A) Traduz diretamente para uma sequência de *macroinstruções* correspondentes
 - Qual é o desempenho e custo do macroprograma?

Nossa ISA

ISA MAC-1 (nível 2)

- 23 Macroinstruções
- de 16 bits (coluna 1)
 - Opcode crescente (4, 8 ou 16 bits), não de tamanho fixo
 - Dois tipos de endereçamento:
 - Endereçamento direto
 - Endereçamento indexado
 - Um operando de 12 bits ou 8 bits ou sem operando
 - Não determina diretamente o número de dados necessário para a operação

Binary	Mnemonic	Instruction	Meaning
0000 xxxxxxxxxxxx	LODD	Load direct	$ac := m[x]$
0001 xxxxxxxxxxxx	STOD	Store direct	$m[x] := ac$
0010 xxxxxxxxxxxx	ADDD	Add direct	$ac := ac + m[x]$
0011 xxxxxxxxxxxx	SUBD	Subtract direct	$ac := ac - m[x]$
0100 xxxxxxxxxxxx	JPOS	Jump positive	If $ac \geq 0$ then $pc := x$
0101 xxxxxxxxxxxx	JZER	Jump zero	If $ac = 0$ then $pc := x$
0110 xxxxxxxxxxxx	JUMP	Jump	$pc := x$
0111 xxxxxxxxxxxx	LOCO	Load constant	$ac := x$ ($0 \leq x \leq 4095$)
1000 xxxxxxxxxxxx	LODL	Load local	$ac := m[sp + x]$
1001 xxxxxxxxxxxx	STOL	Store local	$m[x + sp] := ac$
1010 xxxxxxxxxxxx	ADDL	Add local	$ac := ac + m[sp + x]$
1011 xxxxxxxxxxxx	SUBL	Subtract local	$ac := ac - m[sp + x]$
1100 xxxxxxxxxxxx	JNEG	Jump negative	If $ac < 0$ then $pc := x$
1101 xx xxxxxxxx	JNZE	Jump nonzero	If $ac \neq 0$ then $pc := x$
1110 xx xxxxxxxx	CALL	Call procedure	$sp := sp - 1; m[sp] := pc; pc := x$
1111000 00000000	PSHI	Push indirect	$sp := sp - 1; m[sp] := m[ac]$
1111001 00000000	POPI	Pop indirect	$m[ac] := m[sp]; sp := sp + 1$
1111010 00000000	PUSH	Push onto stack	$sp := sp - 1; m[sp] := ac$
1111011 00000000	POP	Pop from stack	$ac := m[sp]; sp := sp + 1$
1111100 00000000	RETN	Return	$pc := m[sp]; sp := sp + 1$
1111101 00000000	SWAP	Swap ac, sp	$tmp := ac; ac := sp; sp := tmp$
1111110 yyyyyyyy	INSP	Increment sp	$sp := sp + y$ ($0 \leq y \leq 255$)
1111111 yyyyyyyy	DESP	Decrement sp	$sp := sp - y$ ($0 \leq y \leq 255$)

xxxxxxxxxxxx is a 12-bit machine address; in column 4 it is called x .
 yyyyyyyy is an 8-bit constant; in column 4 it is called y .